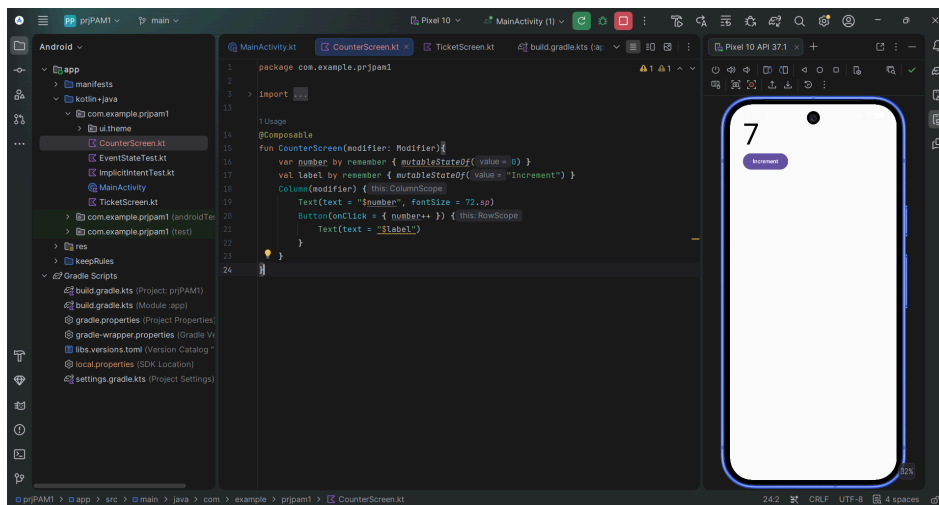


Nama : Janice Kusuma
NIM : 245150400111021
Kelas : PAM-SI-D

JAWABAN MODUL 4

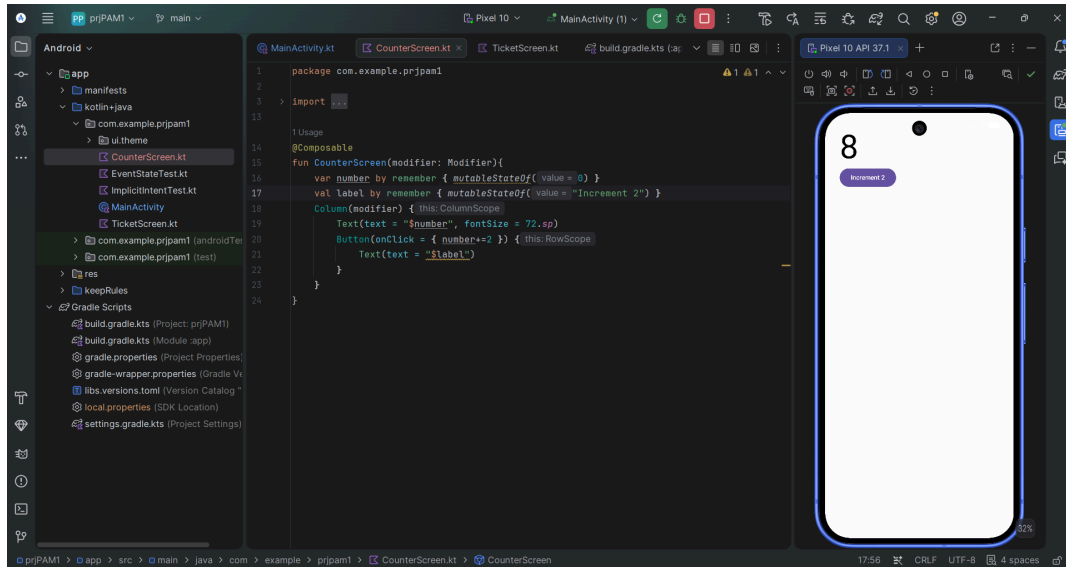
State Hoisting

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.



- a. Apa yang terjadi setiap kali tombol "Increment" diklik?
Setiap kali tombol "Increment" di klik, angka counter di atasnya berubah dan terus bertambah sebanyak kita menekan tombolnya.
- b. Apa yang menyebabkan hal tersebut terjadi?
Karena adanya variabel state yang akan mengubah nilai counter setiap kali state-nya berubah. Variabel state yang dimaksud adalah *number* dan *label*, khususnya *number*, dimana terdapat statement `onClick = { number++ }`.
- c. Variabel *state* apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?
Variabel yang menampilkan perubahan angka pada text adalah variabel *number*, yang didefinisikan pada statement `var number by remember { mutableStateOf(0) }`.
- d. Variabel *state* apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?
Variabel *state* label yang menampilkan label teks, yang didefinisikan pada statement `val label by remember { mutableStateOf("Increment") }`.

2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable **CounterScreen** agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".

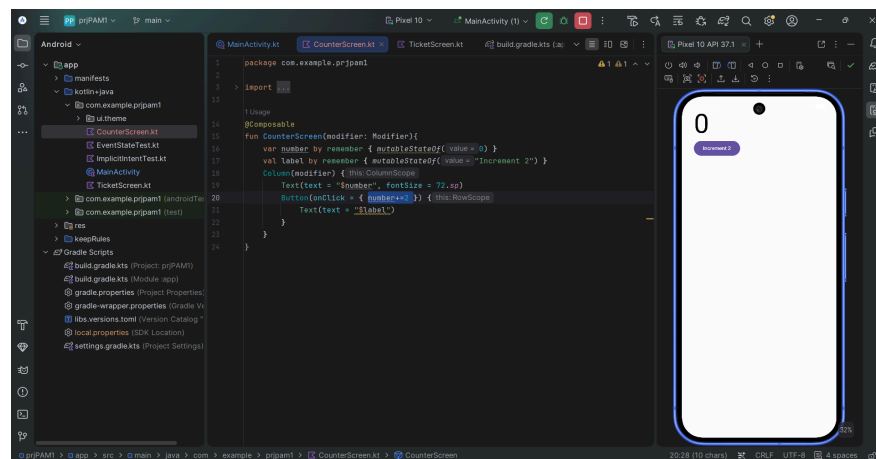


a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

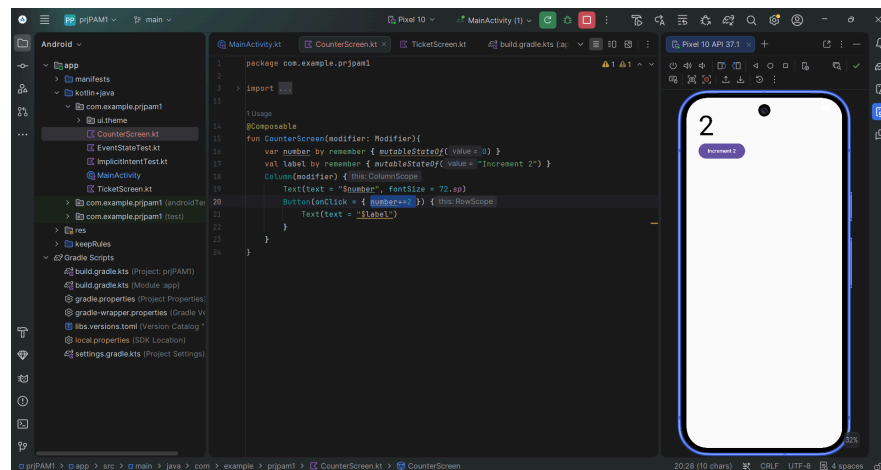
Yang perlu dilakukan adalah mengubah nilai state *number* dan *label*. Mengubah number pada button dan onClick menjadi $number += 2$. Kemudian, mengubah value dari state label menjadi "Increment 2".

b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

Sebelum klik



Sesudah klik



- Langkah selanjutnya adalah melakukan *state* dan *event hoisting* pada state dan event yang sebelumnya terdapat di dalam Composable **CounterScreen**. Ubah signature kode program pada bagian fungsi Composable **CounterScreen** dari yang sebelumnya:

```
fun CounterScreen(modifier: Modifier)
```

menjadi:

```
fun CounterScreen(modifier: Modifier,
                  number: Int, label: String, onClick: () ->
Unit)
```

Ubah *body* di dalam Composable **CounterScreen** menjadi:

```
// var number by remember { mutableStateOf(0) }
// val label by remember { mutableStateOf("Increment") }
Column(modifier) {
    Text(text = "$number", fontSize = 72.sp)
    Button(onClick = onClick) {
        Text(text = "$label")
    }
}
```

Ubah pemanggilan fungsi Composable **CounterScreen** di dalam fungsi **onCreate()** dari yang sebelumnya:

```
CounterScreen(modifier = Modifier.padding(32.dp))
```

menjadi:

```

var number by remember { mutableStateOf(0) }
CounterScreen(Modifier.padding(32.dp),
    number = number,
    label = "Increment",
    onClick = { number++ }
)

```

Perhatikan bahwa deklarasi variabel state `number` dari yang sebelumnya dilakukan di dalam fungsi `Composable CounterScreen`, sekarang dideklarasikan di dalam fungsi `onCreate()` sebelum fungsi `Composable CounterScreen` dipanggil dari dalam fungsi `onCreate()`.

4. Jalankan program, dan kemudian klik beberapa kali tombol **Increment** sehingga angka yang ditampilkan berubah.
 - a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?
Program berjalan dengan baik dan tampilan yang muncul sama persis seperti yang dilakukan pada tahap sebelumnya.
 - b. Di bagian kode program manakah saat ini proses **increment** dilakukan?
Proses **increment** dilakukan pada `main activity`, pada bagian parameter `CounterScreen` yang dimasukkan, dimana jika button di-klik, maka akan dilakukan **increment**. Terdapat pada statement: `onClick = { number++ }`
 - c. Di bagian kode program manakah saat ini state **Composable Text** dan **Composable Button "Increment"** dideklarasikan?
State **composable text** dan **button Increment** dideklarasikan di `MainActivity`, di luar function `CounterScreen`, namun untuk nilai yang digunakan untuk state-nya dimasukkan pada parameter di `CounterScreen`.
5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 (4) dilakukan sedemikian rupa sehingga:
 - a. Label tombol **"Increment"** menjadi **"Double"**
 - b. Nilai awal variabel `number` dari 0 menjadi 1.
 - c. Ubah fungsi/lambda `onClick` pada pemanggilan **Composable CounterScreen** di dalam fungsi `onCreate()` sedemikian rupa sehingga yang dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel `number`.
 - d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi **Composable CounterScreen** di dalam fungsi

onCreate() setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan (tahap no 5).

Bagian yang mendeklarasikan state dan pemanggilan fungsi composable dalam fungsi onCreate adalah bagian berikut.

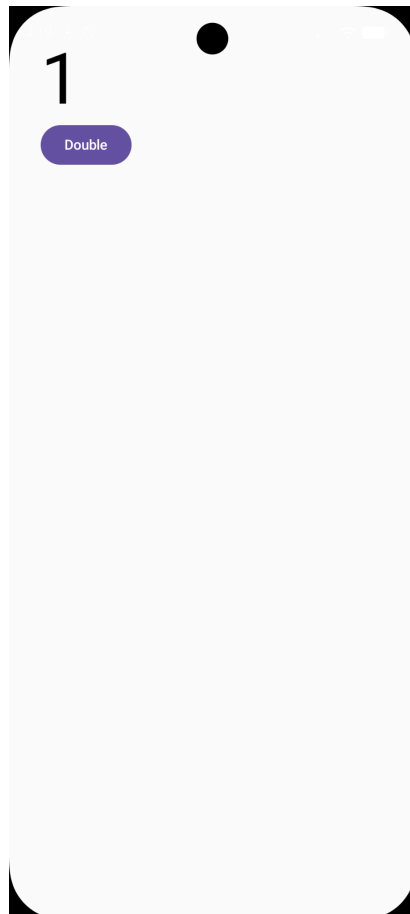
```
var number by remember { mutableStateOf(1) }  
CounterScreen(Modifier.padding(32.dp),  
    number = number,  
    label = "Double",  
    onClick = { number *= 2 }  
)
```

```
var number by remember { mutableStateOf( value = 1) }  
CounterScreen(Modifier.padding( all = 32.dp),  
    number = number,  
    label = "Double",  
    onClick = { number *= 2 }  
)
```

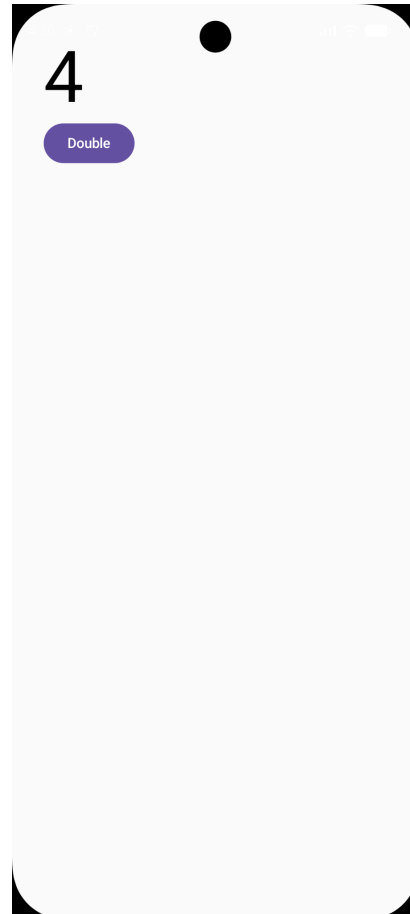
6. Jalankan program,

- a. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.

Sebelum



Sesudah (klik 2 kali)



- b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?**

Percobaan dilakukan pada kode program bagian deklarasi state:

```
var number by remember { mutableStateOf(1) } — 0 diganti 1  
label = "Double", — "Increment" diganti "Double"  
onButtonClick = { number *= 2 } — ++ diganti *= 2
```

- c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.**

Tidak ada perubahan yang dilakukan dari tahap-tahap percobaan sebelumnya pada fungsi composable CounterScreen, yang diubah hanya nilai pada MainActivity saja di parameter fungsi yang dipanggil.

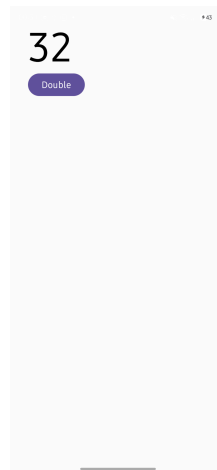
- 7. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan *state hoisting* dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?**

Dari percobaan tersebut dapat disimpulkan bahwa state hoisting membuat komponen UI lebih fleksibel terhadap perubahan kebutuhan fungsional. Sebelumnya, untuk mengubah komponen UI, harus langsung di fungsi komponen tersebut, tapi dengan state hoisting ini, jadi lebih mudah untuk mengubah nilai-nilai komponen yang ada tanpa mengubah struktur internal dari CounterScreen(). Ini juga membuat composable lebih reusable, mudah dipelihara, memiliki single source of truth, serta menerapkan alur data satu arah antara parent dan child.

Perubahan Konfigurasi

- 1. Jalankan program, dengan memastikan posisi perangkat pada posisi *portrait*, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmuka-nya.**

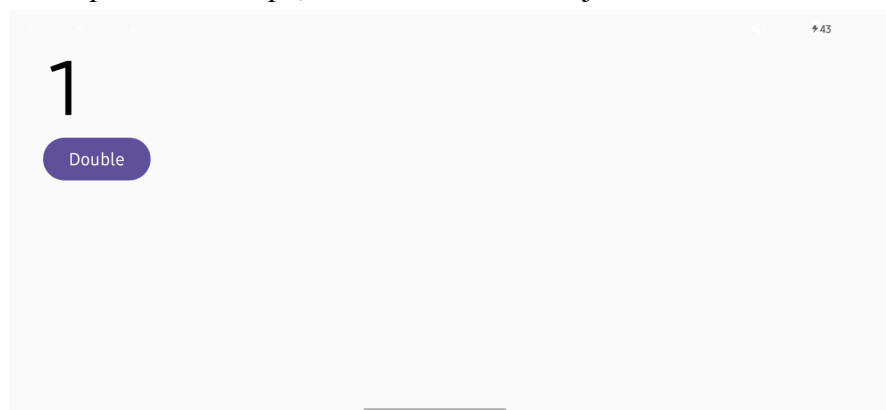
Setelah 5 kali klik tombol "Double", angka yang tampil menjadi 32. Tampilannya seperti berikut.



2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.

a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.

Saat diubah ke posisi landscape, counter berubah menjadi 1 kembali.



b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan adalah 1, bukan 32 seperti hasil yang sudah dilihat sebelumnya. Hal ini dapat terjadi karena `remember` hanya mempertahankan state selama composable masih berada dalam composition dan selama tidak terjadi penghancuran/recreation Activity, sedangkan jika orientasi berubah dari portrait jadi landscape, Android melakukan configuration change yang umumnya menyebabkan Activity dibuat ulang. Akibatnya, activity tidak tersimpan pada remember, dan dibuat ulang dari awal.

Ubah bagian kode program di dalam fungsi `onCreate()` seperti berikut ini:

```
var number by remember { mutableStateOf(1) }
```

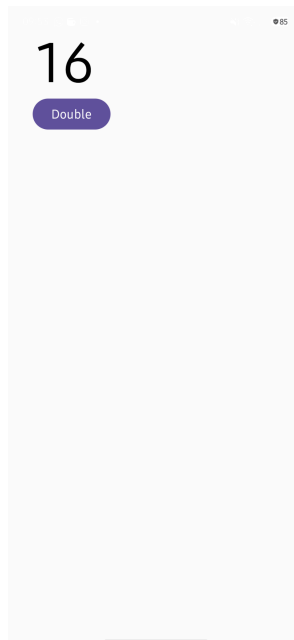
menjadi:

```
var number by rememberSaveable { mutableStateOf(1) }
```

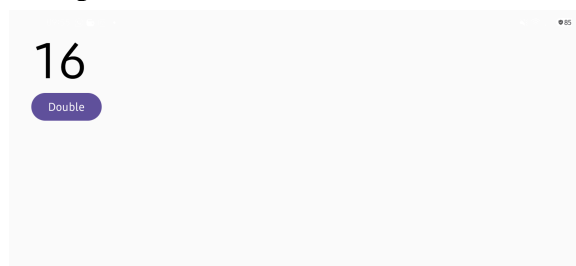
Lakukan *import dependency* atas perubahan konstruk `remember` menjadi `rememberSaveable` yang dilakukan (dengan memposisikan kursor pada konstruk `rememberSaveable` tersebut dan melakukan refactoring dengan menekan kombinasi tombol Alt-Enter di jendela *editor* Android Studio).

Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi *portrait*, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel `number` saat program pertama kali dijalankan. **Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmuka-nya.**

Setelah di klik 4 kali, angka counter menjadi 16, seperti tampilan di bawah ini.



3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi *landscape*.



Setelah diputar menjadi *landscape*, angka tetap 16 dan tidak ter-reset seperti sebelumnya.

b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan adalah 16, karena penggunaan `rememberSaveable` yang menyimpan nilai state tersebut agar dapat dipulihkan ketika Activity dibuat ulang akibat configuration change, sehingga pada saat rotasi, compose dapat mengembalikan nilai sebelumnya.

c. Apa fungsi dari konstruk `rememberSaveable` pada kode program Android yang membedakannya dari konstruk `remember`?

Konstruk `remember` tidak bertahan saat terjadi configuration change, karena state nya hilang saat activity dibuat ulang, dimana `remember` mempertahankan nilai saat recomposition, sedangkan `rememberSaveable` mempertahankan nilai saat recomposition sekaligus ketika terjadi perubahan konfigurasi, sehingga nilai state nya bisa bertahan dan di-compose kembali saat activity dibuat ulang.

4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Contoh kasus nyata penggunaannya ada di Form Pendaftaran (form input pendaftaran). Pada kasus ini, konstruk `rememberSaveable` cocok digunakan agar pengguna tidak selalu mengulang pengisian ketika ada perubahan konfigurasi. Misalnya, jika pengguna sedang mengisi form dan ternyata ponsel-nya dirotasi, form yang telah diisi tidak akan hilang begitu saja (isinya). Lalu, saat pengguna mengubah tema aplikasi (light mode/dark mode), dan saat pengguna berpindah ke aplikasi lain disaat mengisi form, sehingga pengalaman pengguna juga menjadi lebih baik dan nyaman.

LaunchedEffect

PERCOBAAN 1

1. Jalankan program,

a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?

Saat pertama kali aplikasi di-run, tampilan pada layar adalah angka 1 pada counter dan tombol "Double" seperti percobaan sebelumnya, dengan tambahan saat pertama kali aplikasi dijalankan, muncul notifikasi snackbar di bawah dengan pesan "Number 1 is shown".

b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

Karena adanya `LaunchedEffect` yang menjalankan kode saat dikomposisi, sehingga saat baru dijalankan, blok kode pada `LaunchedEffect` akan langsung di run.

c. Apa yang memicu pesan pada `SnackBar` tersebut ditampilkan? Pada bagian kode program yang mana?

Pemicu munculnya pesan pada SnackBar adalah dijalankannya `LaunchedEffect(number)`, dimana adanya state number menjadi pemicu munculnya pesan. Kode program yang dijalankan untuk menampilkan SnackBar adalah bagian `snackbarHostState.showSnackBar("Number $number is shown!")` pada `LaunchedEffect(number)`.

2. Klik button "Double" satu kali,

a. Apa yang terjadi?

Terjadi penggandaan angka counter menjadi 2, dan muncul pesan SnackBar lagi di bawah dengan tulisan "Number 2 is shown!"

b. Mengapa hal tersebut terjadi? Jelaskan!

Hal ini terjadi karena number menjadi key atau pemicu dijalankannya side effect, sehingga saat state number berubah, key number pada `LaunchedEffect` juga berubah, dan akhirnya dijalankan kembali dengan angka terbaru.

PERCOBAAN 2

3. Jalankan program,

a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?

Pertama kali saat program dijalankan, yang tampil di layar adalah angka 1 dengan SnackBar yang sama seperti percobaan sebelumnya.

b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

Setelah menunggu selama 2 detik, tampilan pada layar menjadi 430 pada counter dan muncul SnackBar "Number 430 is shown!"

c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada SnackBar tersebut ditampilkan? Pada bagian kode program yang mana?

Perubahan angka dan pesan yang ditampilkan pada SnackBar bisa terjadi karena adanya `LaunchedEffect` lain yaitu `Repo.getData()`, dimana fungsi tersebut terdapat delay selama 2 detik dan mengembalikan nilai random antara 100 sampai 1000. Oleh karena itu, muncul angka random di aplikasi yang dijalankan, yaitu 430. Bagian kodenya adalah

```
LaunchedEffect(Unit) {  
    number = Repo.getData()  
}
```

4. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,

a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

`LaunchedEffect(state)` menggunakan state sebagai key, sedangkan `LaunchedEffect(Unit)` menggunakan unit sebagai key, kemudian `LaunchedEffect(state)` akan berjalan kembali ketika nilai state (key) nya berubah, sedangkan `LaunchedEffect(Unit)` tidak berjalan kembali jika state lain berubah, hanya

berjalan sekali saja. Contohnya pada percobaan, `LaunchedEffect(Unit)` yang mengembalikan nilai random itu hanya dijalankan sekali dan tidak terjadi lagi walaupun tombol double di-klik.

b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

`LaunchedEffect(state)` dijalankan ketika pertama kali composable masuk ke composition dan setiap kali nilai state yang menjadi key berubah. `LaunchedEffect(Unit)` hanya dijalankan ketika pertama kali composable masuk ke composition saja karena unit merupakan key tetap yang nilainya tidak berubah seperti state.